

What is the Best Sorting Algorithm?

The Optimization of Time in Data Sorting

Jeremy Roalef

CITA 225: Introduction to Data Structure

State University of New York at Morrisville

Professor Elfituri

May 6, 2024

Table of Contents

Introduction	3
Methodology.....	5
Results.....	6
Insertion Sort	6
Bubble Sort.....	6
Merge Sort	7
Analysis	8
Conclusion.....	12
Figure 1. Sort Complexity for Insertion Sort Using Random Order.....	8
Figure 2. Sort Complexity for Bubble Sort Using Random Order.....	9
Figure 3. Sort Complexity for Merge Sort Using Random Order	10
Figure 4. Comparison of Long-Run Behavior of Sort Algorithms Using Random Order	11
Table 1. Insertion sort runtime over ten trials.....	6
Table 2. Bubble sort runtime over ten trials.....	7
Table 3. Merge sort runtime over ten trials.....	7

Introduction

A program will make use of many data types known as elements. Common elements are Booleans, strings like characters and sentences, and numbers like integers, floats, and doubles. These elements may be stored in containers known as data structures to efficiently and effectively store and retrieve information through algorithmic designs. Standard algorithms include inserting, removing, searching, and sorting. This paper will discuss standard sorting algorithms.

Sort algorithms work by analyzing the contents inside a data structure and moving them around according to a given condition. For example, a common sorting condition is from smallest to largest. Assume a list of ten numbers is randomly ordered. Running a sort algorithm conditioned from smallest to largest will make the first element the smallest value, the second element the second smallest value, etc. A sorted data structure makes it easier for the analyst to read the contents. However, sorting algorithms are time-consuming.

There are many types of sorting algorithms. This project focuses on three sorting algorithms:

- **Insertion sort.** Insertion sort works by taking one element at a time out of the data structure and appending it to a new, sorted data structure.
- **Bubble sort.** Bubble sort works by analyzing adjacent elements in the data structure and flipping them if they fulfill the sorting condition. For example, suppose you have a data structure with elements [2,1]. Using a sequential condition, the

bubble sort algorithm will read the contents, realize the contents are not in order, and flip the two elements. The revised data structure will result in [1,2].

- **Merge sort.** Merge sort works by splitting the data structure into two smaller data structures until the data structures only contain one element. The data structures' contents are “merged” and sorted from there.

All sort algorithms perform the same task but differ in sort complexity. Sort complexity is the expected time of completion that an algorithm will take, based on how the algorithm is programmed.

This project aims to find evidence of a faster sorting algorithm by analyzing the sort complexity of insertion, bubble, and merge sort. This paper seeks to prove that the expected sort complexity for insertion sort, bubble sort, and merge sort is accurate and prove which of the three sort algorithms takes the least time to run.

Methodology

The three algorithms were downloaded and run on Python using a personal laptop. Within a trial, each algorithm ran nine times with different sort sizes: small ($n = 1,000$), medium ($n = 10,000$), and large ($n = 100,000$). For each sort size, the algorithm was tested with a different order of elements in the data structure: sorted, reverse-sorted, and random. The sorted data structure contains elements already sorted by the sort condition (i.e., 1,2,3,4,5), which can be considered the best-case scenario. The reverse-sorted data structure contains elements sorted opposite to the sort condition (i.e., 5,4,3,2,1), which can be viewed as the worst-case scenario. The random data structure is the most likely case scenario, where the elements in the data structure are a mesh of partially sorted and unsorted (i.e., 1,2,5,3,4).

The personal laptop was left running for 25-minute intervals to run the algorithms. The display settings were altered to keep the computer running for thirty minutes to prevent it from entering sleep mode. All applications except for Python on the laptop were shut down so the program could run at maximum efficiency.

There are three variables to consider in this experiment: the sort algorithms, sort order, and sort size. These three variables were tested in each trial. After each session, the sorting complexity was recorded in a table. This procedure ran for a total of ten trials.

Results

Insertion Sort

The insertion sort algorithm ran quickly for data structures already sorted. The average runtimes were 0.003 seconds for small input sizes, 0.0005 seconds for medium input sizes, and 0.0088 seconds for large input sizes (See Table 1). However, the same cannot be said for reverse-sorted and random-sort orders. For reverse sorted order, the algorithm took averages of 0.0281 seconds, 2.9170 seconds, and 302.4173 seconds for small, medium, and large input sizes, respectively. For random sort order, the algorithm took averages of 0.0139 seconds, 1.5335 seconds, and 147.5437 seconds for small, medium, and large input sizes, respectively.

Table 1. Insertion sort runtime over ten trials.

Insertion Sort												
Input Size	Sort Order	Runtime 1 (s)	Runtime 2 (s)	Runtime 3 (s)	Runtime 4 (s)	Runtime 5 (s)	Runtime 6 (s)	Runtime 7 (s)	Runtime 8 (s)	Runtime 9 (s)	Runtime 10 (s)	Average Runtime (s)
Small	Sorted	0.0000	0.0000	0.0000	0.0014	0.0000	0.0000	0.0000	0.0000	0.0013	0.0000	0.0003
	Reverse Sorted	0.0280	0.0244	0.0422	0.0237	0.0251	0.0245	0.0273	0.0257	0.0295	0.0302	0.0281
	Random	0.0134	0.0132	0.0229	0.0115	0.0130	0.0119	0.0136	0.0129	0.0150	0.0118	0.0139
Medium	Sorted	0.0000	0.0000	0.0010	0.0000	0.0010	0.0010	0.0010	0.0011	0.0000	0.0000	0.0005
	Reverse Sorted	2.6195	2.6156	5.0279	2.6131	2.7109	2.6253	2.6022	2.6006	3.1094	2.6453	2.9170
	Random	1.9688	2.2294	1.8670	1.2558	1.3012	1.3583	1.2792	1.2815	1.5225	1.2709	1.5335
Large	Sorted	0.0127	0.0137	0.0110	0.0070	0.0070	0.0070	0.0080	0.0060	0.0080	0.0078	0.0088
	Reverse Sorted	420.6696	457.0204	279.5341	256.8288	263.5107	260.3727	261.3278	257.9964	306.7895	260.1227	302.4173
	Random	177.9660	245.9049	126.2309	125.8408	130.1016	136.4107	127.7017	126.9561	150.7922	127.5326	147.5437

Bubble Sort

The bubble sort algorithm ran similarly to the insertion sort algorithm but took longer for larger input sizes. Sorted data structures took averages of 0.0202 seconds, 2.0967 seconds, and 198.4459 seconds for small, medium, and large input sizes, respectively (See Table 2). For reverse sorted order, the algorithm took averages of 0.0412 seconds, 4.5266 seconds, and 415.8817 seconds for small, medium, and large input sizes,

respectively. For random sort order, the algorithm took averages of 0.0351 seconds, 3.5488 seconds, and 347.4247 seconds for small, medium, and large input sizes, respectively.

Table 2. Bubble sort runtime over ten trials.

Bubble Sort												
Input Size	Sort Order	Runtime 1 (s)	Runtime 2 (s)	Runtime 3 (s)	Runtime 4 (s)	Runtime 5 (s)	Runtime 6 (s)	Runtime 7 (s)	Runtime 8 (s)	Runtime 9 (s)	Runtime 10 (s)	Average Runtime (s)
Small	Sorted	0.0175	0.0377	0.0180	0.0177	0.0185	0.0180	0.0180	0.0178	0.0202	0.0187	0.0202
	Reverse Sorted	0.0376	0.0642	0.0376	0.0371	0.0381	0.0381	0.0372	0.0384	0.0444	0.0397	0.0412
	Random	0.0303	0.0670	0.0307	0.0306	0.0301	0.0327	0.0312	0.0318	0.0346	0.0317	0.0351
Medium	Sorted	1.8772	3.8569	1.8980	1.8997	1.8763	1.9523	1.8606	1.8823	1.9611	1.9030	2.0967
	Reverse Sorted	4.0326	8.1271	4.0210	4.0611	4.0801	4.2818	4.0273	4.0733	4.5241	4.0374	4.5266
	Random	3.2571	5.7671	3.1944	3.2595	3.2569	3.3528	3.2971	3.2450	3.5543	3.3038	3.5488
Large	Sorted	191.7501	225.2118	190.2282	190.5457	191.0321	199.2849	199.7411	191.5158	202.1203	203.0286	198.4459
	Reverse Sorted	405.2212	404.1887	402.8653	405.4722	405.7885	423.5508	423.1290	405.3266	459.8702	423.4044	415.8817
	Random	340.8319	344.0596	338.9363	342.1434	342.7922	351.0473	346.3629	343.7640	376.2208	348.0888	347.4247

Merge Sort

The merge sort algorithm ran fast. The average runtimes for sorted data structures were 0.0010 seconds, 0.0121 seconds, and 0.1507 seconds for small, medium, and large input sizes, respectively (See Table 3). For reverse sorting, the average runtimes were 0.0009 seconds, 0.0128 seconds, and 0.1527 seconds for small, medium, and large input sizes, respectively. For random order, the algorithm took averages of 0.0011 seconds, 0.0151 seconds, and 0.1916 seconds for small, medium, and large input sizes, respectively.

Table 3. Merge sort runtime over ten trials.

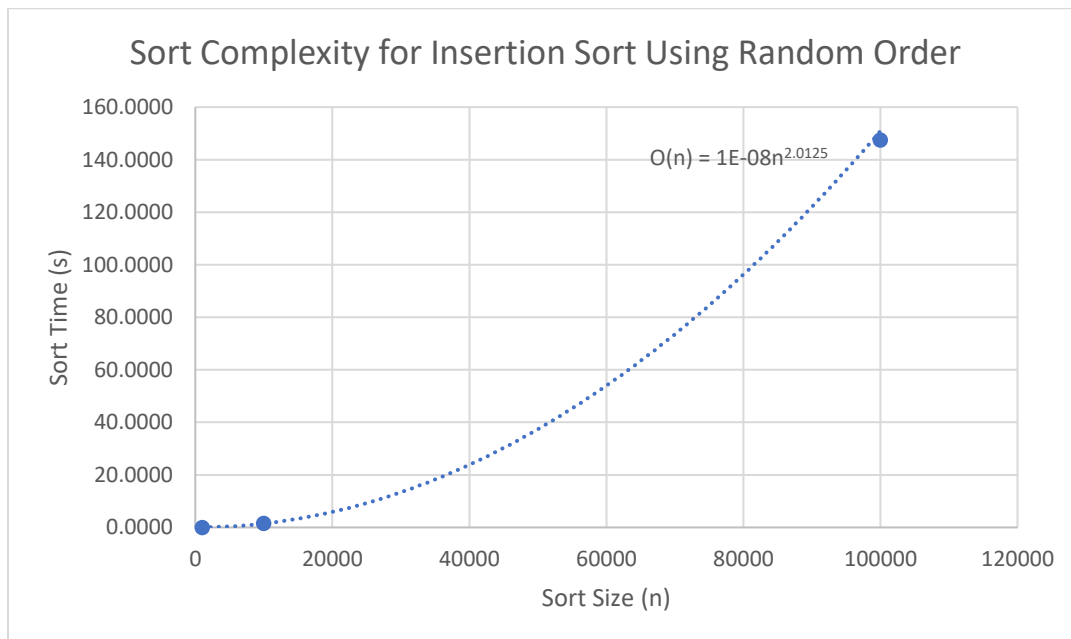
Merge Sort												
Input Size	Sort Order	Runtime 1 (s)	Runtime 2 (s)	Runtime 3 (s)	Runtime 4 (s)	Runtime 5 (s)	Runtime 6 (s)	Runtime 7 (s)	Runtime 8 (s)	Runtime 9 (s)	Runtime 10 (s)	Average Runtime (s)
Small	Sorted	0.0010	0.0010	0.0010	0.0000	0.0010	0.0010	0.0010	0.0010	0.0010	0.0020	0.0010
	Reverse Sorted	0.0010	0.0010	0.0010	0.0010	0.0010	0.0000	0.0010	0.0010	0.0010	0.0010	0.0009
	Random	0.0010	0.0010	0.0010	0.0010	0.0011	0.0010	0.0010	0.0010	0.0020	0.0010	0.0011
Medium	Sorted	0.0120	0.0136	0.0120	0.0120	0.0117	0.0120	0.0121	0.0120	0.0122	0.0117	0.0121
	Reverse Sorted	0.0125	0.0140	0.0120	0.0120	0.0130	0.0130	0.0122	0.0130	0.0126	0.0129	0.0128
	Random	0.0150	0.0151	0.0150	0.0160	0.0156	0.0150	0.0146	0.0140	0.0154	0.0151	0.0151
Large	Sorted	0.1509	0.1593	0.1485	0.1490	0.1472	0.1504	0.1557	0.1460	0.1545	0.1457	0.1507
	Reverse Sorted	0.1539	0.1619	0.1499	0.1540	0.1505	0.1539	0.1498	0.1524	0.1504	0.1507	0.1527
	Random	0.1906	0.1975	0.1849	0.1925	0.1895	0.1945	0.1899	0.1905	0.1931	0.1926	0.1916

Analysis

This paper aims to prove that the expected sort complexity of the sort algorithms is accurate. By plotting the average runtime of random sort order scenarios on a graph using small, medium, and large input sizes, we can calculate the line of best fit for each sort algorithm. All calculations were made using Excel Spreadsheet. When comparing the expected sort complexity to the observed sort complexity, we analyze the highest degree of the equation.

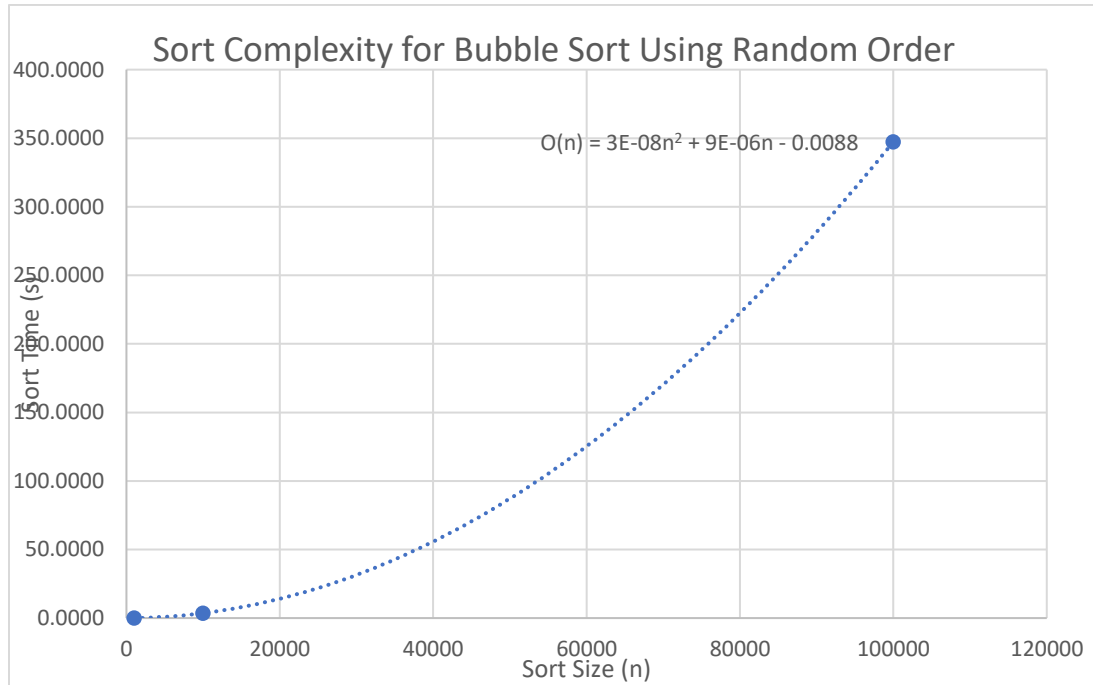
The observed sort complexity of the insertion sort algorithm, using random order, is $O(n) = n^{2.0125}$ (see Figure 1). Given the expected sort complexity of $O(n) = n^2$, our observed sort complexity is very close to what is expected.

Figure 1. Sort Complexity for Insertion Sort Using Random Order



For the bubble sort algorithm, the observed sort complexity is $O(n) = n^2$ (see Figure 2), which matches the expected sort complexity $O(n) = n^2$.

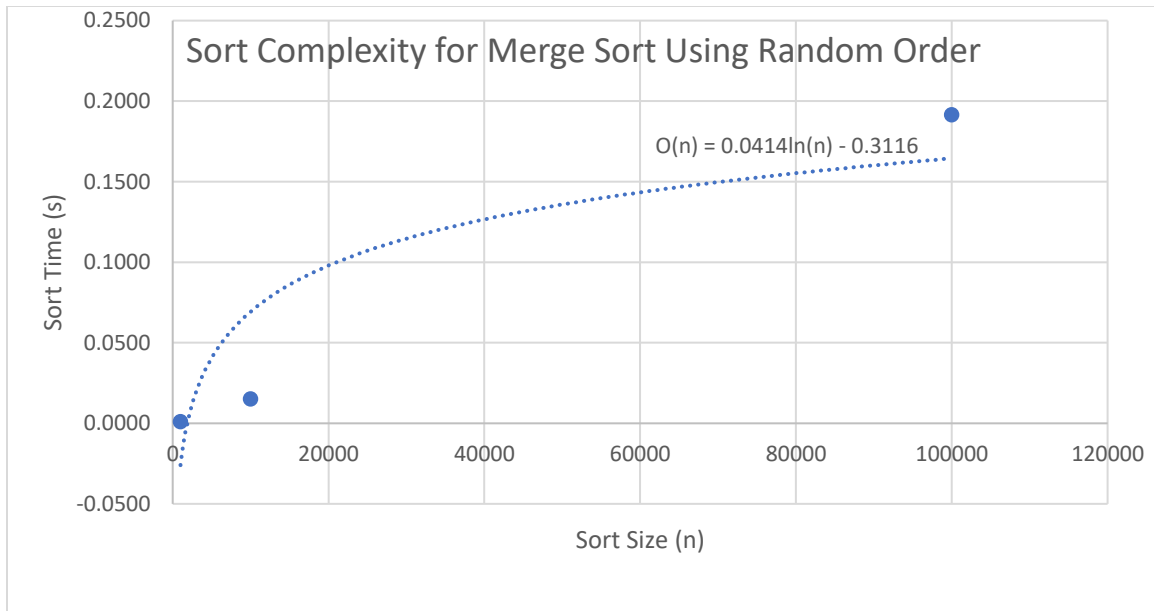
Figure 2. Sort Complexity for Bubble Sort Using Random Order



For merge sort, the observed sort complexity is $O(n) = \ln(n)$ (see Figure 3).

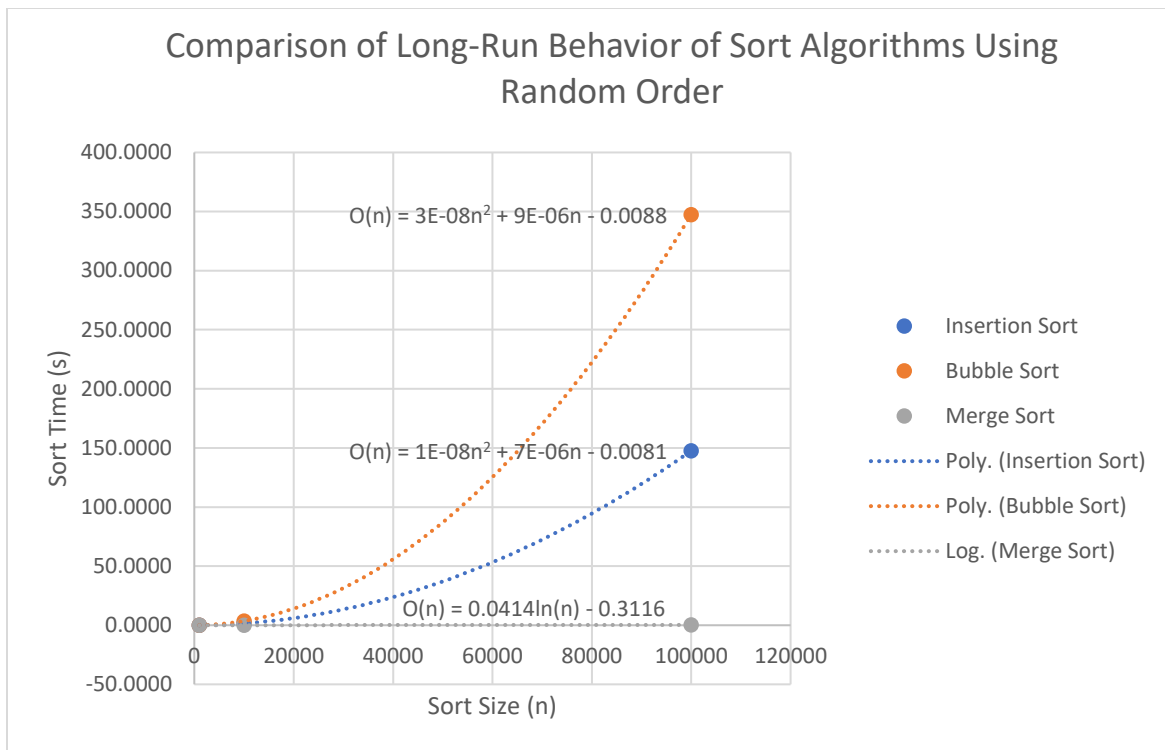
Rewriting the equation as a logarithm of base 10, we get $O(n) = \log_{10} n$, which is similar to the expected sort complexity $O(n) = n \log_{10} n$.

Figure 3. Sort Complexity for Merge Sort Using Random Order



Analyzing the long-run behavior of the algorithms' sort complexity, we can conclude which of the three sort algorithms takes the least amount of time (See Figure 4). The sort algorithm that takes the longest is the bubble sort algorithm, with insertion sort coming in second. The fastest sort algorithm is merge sort.

Figure 4. Comparison of Long-Run Behavior of Sort Algorithms Using Random Order



Conclusion

Sort algorithms perform the same task, but they differ in their sort complexity. Based on how the algorithm is set up, the sort complexity may increase or decrease. The goal of analyzing the insertion sort, bubble sort, and merge sort algorithms was to determine if the expected sort complexity of the algorithm matched the observed sort complexity and find which sort algorithm takes the least time to run. Analyzing the long-run behavior of the sort algorithms, we can conclude that the observed insertion and bubble sort complexities are very close to their expected counterparts, and the observed merge sort complexity is similar to the expected sort complexity. Analyzing the long-run behavior of the insertion, bubble, and merge sort complexities shows that the merge sort algorithm takes the least amount of time given a large set of data.

Some restrictions of this test are the number of sort sizes used to calculate sort complexity. Each sort algorithm used three different sort sizes. For future experimentation, it would be worthwhile to use more sort sizes to better estimate the sort complexity of the graphs.